

R Programming — Complete Questions & Answers

Experiments 1 to 4

EXPERIMENT 1: Basic Operations and Data Handling in R

MAIN PROBLEM STATEMENT

Obtain an integer input (say n) from the user. Then perform the following:

Question (a): Define a function to find the factorial of n .

Answer:

```
n <- as.integer(readline("Enter an integer n: "))

factorial_func <- function(n) {
  if (n == 0 || n == 1) return(1)
  return(n * factorial_func(n - 1))
}

cat("Factorial of", n, "is:", factorial_func(n), "\n")

For n=5: factorial_func(5) → 120
```

Question (b): Define a function to generate the Fibonacci series of length n .

Answer:

```
fibonacci <- function(n) {
  fib <- numeric(n)
  fib[1] <- 0
```

```

    if (n >= 2) fib[2] <- 1
    for (i in 3:n) fib[i] <- fib[i-1] + fib[i-2]
    return(fib)
}
cat("Fibonacci series of length", n, ":\n")
print(fibonacci(n))

```

For n=8: 0 1 1 2 3 5 8 13

Question (c): Use `colours()` as an object to assign first `n` colours to a vector `my.colours`.

Answer:

```

all_colours <- colours()
my.colours <- all_colours[1:n]
cat("First", n, "colours assigned to my.colours\n")

```

Question (d): Access all the elements of `my.colours` from second-last position to second position.

Answer:

```

result_d <- my.colours[(length(my.colours)-1):2]
print(result_d)

```

R indexing is 1-based. `(length-1)` gives the second-last element and 2 gives the second element.

Question (e): Access all but the first and third element of `my.colours`.

Answer:

```

result_e <- my.colours[-c(1, 3)]
print(result_e)

```

Question (f): Sort `my.colours` in descending order.

Answer:

```

sorted_colours <- sort(my.colours, decreasing = TRUE)
print(sorted_colours)

```

Question (g): At which index is the colour “azure”? Use both `which()` and `grep()` function.

Answer:

```
# Using which()
idx_which <- which(my.colours == "azure")
cat("Index using which():", idx_which, "\n")

# Using grep()
idx_grep <- grep("^azure$", my.colours)
cat("Index using grep():", idx_grep, "\n")

grep() uses regex. ^azure$ matches exactly 'azure'. which()
matches exact value.
```

Question (h): Replace all instances of “antique” with “modern” in `my.colours`.

Answer:

```
my.colours <- gsub("antique", "modern", my.colours)
cat("After replacement:\n")
print(my.colours)
```

EXTRA PRACTICE ASSIGNMENTS — Experiment 1

Assignment 1: Generate a sequence from 210 to 215 in steps of 0.8. Find values greater than 211 and less than 213. Repeat the sequence twice with each element repeated 3 times, and display sorted from largest to smallest.

Answer:

```
# Generate sequence
s <- seq(210, 215, by = 0.8)
cat("Sequence:", s, "\n")

# Values > 211 and < 213
filtered <- s[s > 211 & s < 213]
cat("Values between 211 and 213:", filtered, "\n")

# Repeat twice, each element 3 times
repeated <- rep(s, times = 2, each = 3)
```

```
cat("Repeated sequence:\n"); print(repeated)

# Sort descending
cat("Sorted descending:\n"); print(sort(repeated, decreasing = TRUE))
```

Assignment 2: Convert the temperatures {45, 77, 20, 19, 101, 120, 212} from Fahrenheit to Celsius using the equation $C = (5/9)(F - 32)$.

Answer:

```
F_temps <- c(45, 77, 20, 19, 101, 120, 212)
C_temps <- (5/9) * (F_temps - 32)
cat("Celsius temperatures:\n")
print(C_temps)
```

Result: 7.22 25.00 -6.67 -7.22 38.33 48.89 100.00

Assignment 3: Write a function for bubble sorting of data.

Answer:

```
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]; x[j] <- x[j+1]; x[j+1] <- temp
      }
    }
  }
  return(x)
}
cat("Bubble sorted:", bubble_sort(c(64, 25, 12, 22, 11)), "\n")
```

Assignment 4: Matrix Operations on a 4×2 matrix A filled row-wise with {4.3, 3.1, 8.2, 8.2, 3.2, 0.9, 1.6, 6.5}.

(a) Construct and store the 4×2 matrix A.

```
A <- matrix(c(4.3, 3.1, 8.2, 8.2, 3.2, 0.9, 1.6, 6.5),
            nrow = 4, ncol = 2, byrow = TRUE)
print(A)
```

(b) Confirm dimensions are 3×2 if you remove any one row.

```

A_minus_row <- A[-1, ]
cat("Dimensions after removing row 1:", dim(A_minus_row), "\n")

(c) Overwrite the second column of A with that same column sorted
smallest to largest.

A[, 2] <- sort(A[, 2])
print(A)

(d) Delete the fourth row and the first column from (c); ensure result
is a single-column matrix.

result_d <- matrix(A[-4, -1], ncol = 1)
print(result_d)

(e) Store the bottom four elements of (c) as a new 2×2 matrix.

result_e <- matrix(A[3:4, ], nrow = 2, ncol = 2)
print(result_e)

(f) Overwrite positions (4,2), (1,2), (4,1), (1,1) with  $-1/2$  of the two
values on the diagonal of (e).

diag_vals <- diag(result_e)
half_neg <- -1/2 * diag_vals
A[4,2] <- half_neg[1]; A[1,2] <- half_neg[2]
A[4,1] <- half_neg[1]; A[1,1] <- half_neg[2]
print(A)

```

Assignment 5: For the diagonal matrix $A = \text{diag}(2,3,5,-1)$, confirm that $A^{-1} \cdot A - I$ gives a 4×4 null matrix.

Answer:

```

A_mat <- matrix(c(2,0,0,0, 0,3,0,0, 0,0,5,0, 0,0,0,-1),
               nrow = 4, byrow = TRUE)
A_inv <- solve(A_mat)
I4 <- diag(4)
result <- A_inv %*% A_mat - I4
cat("A_inv * A - I4 (should be null matrix):\n")
print(round(result, 10))

```

Assignment 6: Create the given data frame and invert Sex for all individuals.

Answer:

```
df <- data.frame(
  Name = c("Alex", "Lilly", "Mark", "Oliver", "Martha", "Lucas", "Caroline"),
  Age = c(25, 31, 23, 52, 76, 49, 26),
  Height = c(117, 163, 190, 179, 163, 183, 164),
  Weight = c(57, 69, 83, 75, 70, 83, 53),
  Sex = c("F", "F", "M", "M", "F", "M", "F"),
  stringsAsFactors = FALSE
)
# Invert sex
df$Sex <- ifelse(df$Sex == "M", "F", "M")
print(df)
```

Assignment 7: Create and store dframe, then perform operations.

(a) Create dframe with person (character), sex (factor: F, M), funny (factor: Low, Med, High).

```
dframe <- data.frame(
  person = c("Stan", "Francine", "Steve", "Roger", "Hayley", "Klaus"),
  sex = factor(c("M", "F", "M", "M", "F", "M"), levels = c("F", "M")),
  funny = factor(c("High", "Med", "Low", "High", "Med", "Med"),
    levels = c("Low", "Med", "High")),
  stringsAsFactors = FALSE
)
```

(b) Append age column: Stan & Francine=41, Steve=15, Roger=1600, Hayley=21, Klaus=60. Reorder columns.

```
ages <- c(41, 41, 15, 1600, 21, 60)
dframe$age <- ages
dframe <- dframe[, c("person", "age", "sex", "funny")]
print(dframe)
```

(c) Create dframe.2 and merge with dframe. Store as dframe.final.

```
dframe.2 <- data.frame(
  person = c("Peter", "Lois", "Meg", "Chris", "Stewie"),
  age = c(42, 40, 17, 14, 1),
  sex = factor(c("M", "F", "F", "M", "M"), levels=c("F", "M")),
  funny = factor(c("High", "High", "Low", "Med", "High"), levels=c("Low", "Med", "High")),
  stringsAsFactors = FALSE
)
dframe.final <- merge(dframe, dframe.2, all = TRUE)
print(dframe.final)
```

(d) Extract names and ages of female individuals with funniness Med OR High.

```
result_d <- dframe.final[dframe.final$sex == "F" &
                        dframe.final$funny %in% c("Med", "High"),
                        c("person", "age")]
print(result_d)
```

(e) Extract all records from `dframe.final` whose names start with 'S'.

```
result_e <- dframe.final[substr(dframe.final$person, 1, 1) == "S", ]
print(result_e)
```

THEORY QUESTIONS — Experiment 1

Q1. What are features of R programming language?

Answer:

R is an open-source, interpreted, functional, and object-oriented language designed for statistical computing and data analysis. Key features:

- **Free and open-source** under GNU GPL
 - **Highly extensible** through thousands of CRAN packages (18,000+)
 - **Powerful built-in** statistical and graphical capabilities
 - **Core data structures:** vectors, matrices, data frames, and lists
 - **Platform-independent:** runs on Windows, macOS, and Linux
 - **Interactive console (REPL)** for rapid prototyping and exploration
-

Q2. What are the advantages of R programming language?

Answer:

- Comprehensive statistical library (base R + 18,000+ packages)
 - Excellent data visualization tools (ggplot2, base graphics, plotly)
 - Active community and extensive documentation
 - Integrates with Python, C++, SQL, and cloud platforms
 - Supports reproducible research via R Markdown / Quarto
 - Free to use and modify — no licensing cost
-

Q3. Explain operators in R programming.

Answer:

R supports several categories of operators:

Type	Operators
Arithmetic	+, -, *, /, ^, %% (modulo), %/% (integer division)
Relational	<, >, ==, !=, <=, >=
Logical	&, , ! (element-wise); &&, (scalar)
Assignment	<-, =, ->
Special	%in% (membership), %*% (matrix multiplication)

Q4. Write about type conversion in R.

Answer:

R performs **implicit coercion** in the order: `logical` → `integer` → `double` → `complex` → `character`.

Explicit conversion is done with:

```
as.integer(x)    # converts to integer
as.numeric(x)    # converts to double
as.character(x)  # converts to string
as.logical(x)    # converts to TRUE/FALSE
as.factor(x)     # converts to factor
```

Example: `as.integer("42")` returns 42; `as.numeric(TRUE)` returns 1.

Q5. Write about scalars in R programming.

Answer:

R has **no true scalar type**. A ‘scalar’ is simply a vector of length 1. For example, `x <- 5` creates a numeric vector of length 1. You can verify this with:

```
is.vector(5)     # TRUE
length(5)        # 1
```

This means all scalar operations in R are actually vector operations on length-1 vectors, making R inherently vectorized.

Q6. Explain `seq()` and `rep()` functions in creating vectors.

Answer:

`seq()` — generates sequences:


```
seq(1, 10, by=2)           # 1 3 5 7 9 (step of 2)
seq(0, 1, length.out=5)    # 0.00 0.25 0.50 0.75 1.00 (5 equally spaced values)
seq(1, 10, by=3)           # 1 4 7 10
```

rep() — repeats elements:

```
rep(c(1,2), times=3)       # 1 2 1 2 1 2 (repeat whole vector 3 times)
rep(c(1,2), each=3)        # 1 1 1 2 2 2 (repeat each element 3 times)
rep(c(1,2), times=2, each=3) # combines both
```

Q7. Explain about sub-setting methods in R programming.

Answer:

R has three main sub-setting operators:

Operator	Behaviour
[]	Returns same type (vector or list); used for one or more elements
[[]]	Extracts a single element from a list or data frame column
\$	Accesses named element of a list or data frame column

For **vectors**, four sub-setting styles:

```
x <- c(10, 20, 30, 40, 50)
x[x > 25]           # logical: 30 40 50
x[2:4]              # index: 20 30 40
x[-1]               # negative: 20 30 40 50 (all except first)
names(x) <- letters[1:5]
x["c"]              # named: 30
```

CASE STUDY — Experiment 1: New Zealand Political Survey

Part (a): Create character vectors **sex** and **party** with correct positions.

Answer:

```

# sex vector: 20 people, default Male
sex <- rep("M", 20)
female_idx <- c(1, 5, 6, 7, 12, 14, 15, 16)
sex[female_idx] <- "F"

# party vector: default National
party <- rep("National", 20)
party[c(1, 4, 12, 15, 16, 19)] <- "Labour"
party[c(6, 9, 11)] <- "Greens"
party[c(10, 20)] <- "Other"

```

Part (b): Create factor vectors. Does `ordered=TRUE` make sense?

Answer:

```

sex_factor <- factor(sex)
party_factor <- factor(party)

```

`ordered=TRUE` makes sense only when there is a natural rank ordering between levels. For **sex** (M/F), there is no natural order — so `ordered=FALSE` is appropriate. For **party**, political parties do not have a universal left-to-right ordering everyone agrees on, so `ordered=FALSE` is also appropriate here. R alphabetically arranges levels by default.

Part (c): Factor sub-setting.

(1) Party choices for only male participants:

```

male_parties <- party_factor[sex_factor == "M"]
print(male_parties)

```

(2) Genders of those who chose National:

```

national_sex <- sex_factor[party_factor == "National"]
print(national_sex)

```

Part (d): Add 6 more people and combine with original factors.

Answer:

```

new_party <- factor(c("National", "Maori", "Maori", "Labour", "Greens", "Labour"),
  levels = levels(party_factor))
new_sex <- factor(c("M", "M", "F", "F", "F", "M"), levels = levels(sex_factor))

```

```
combined_party <- c(party_factor, new_party)
combined_sex <- c(sex_factor, new_sex)
```

Part (e): Create a confidence factor with levels Low [0,30], Moderate (30,70], High (70,100].

Answer:

```
confidence_vals <- c(93,55,29,100,52,84,56,0,33,52,35,53,55,46,
                    40,40,56,45,64,31,10,29,40,95,18,61)
conf_factor <- cut(confidence_vals,
                  breaks = c(-1, 30, 70, 100),
                  labels = c("Low", "Moderate", "High"))
```

Part (f): Extract confidence levels for Labour and National identifiers.

Answer:

```
conf_labour <- conf_factor[party == "Labour"]
conf_national <- conf_factor[party == "National"]

cat("Labour confidence levels:\n"); print(conf_labour)
cat("National confidence levels:\n"); print(conf_national)
```

Observation: Labour identifiers tend to have higher confidence levels (Moderate to High) that Labour will win more seats than National, which is expected. National identifiers tend to show Low or Moderate confidence.

EXPERIMENT 2: Data Visualization in R

MAIN PROBLEM STATEMENT

Question (a–e): Using the iris dataset — Create a scatter plot of petal width vs petal length, classify species using given rules, distinguish species visually, create a sepal length boxplot, and stack both figures in a PDF.

Answer (Complete Code):

```
# Setup
data(iris)

# Classify species by petal width and petal length rules
classify_species <- function(pw, pl) {
  if (pw >= 0 & pw < 0.75 & pl >= 0 & pl < 2.5) return("Setosa")
  if (pw >= 0.75 & pw < 1.75 & pl >= 2.5 & pl < 5) return("Versicolour")
  if (pw >= 1.75 & pw < 3 & pl >= 5 & pl < 7) return("Virginica")
  return("Unknown")
}
iris$Classified <- mapply(classify_species,
                         iris$Petal.Width, iris$Petal.Length)

# Colors and point characters per species
species_colors <- c("Setosa" = "firebrick",
                    "Versicolour" = "steelblue",
                    "Virginica" = "forestgreen")
species_pch <- c("Setosa" = 16, "Versicolour" = 17, "Virginica" = 15)

col_vec <- species_colors[iris$Classified]
pch_vec <- species_pch[iris$Classified]

# Open PDF output device
pdf("iris_plots.pdf", width = 8, height = 10)
par(mfrow = c(2, 1), mar = c(5, 5, 4, 2))

# Figure 1: Scatter plot - Petal Width vs Petal Length
plot(iris$Petal.Length, iris$Petal.Width,
     col = col_vec,
     pch = pch_vec,
     xlab = "Petal Length (cm)",
     ylab = "Petal Width (cm)",
     main = "Figure 1: Iris - Petal Width vs Petal Length",
     cex = 1.3, cex.axis = 1.1, cex.lab = 1.2, cex.main = 1.3,
     las = 1)
grid(col = "grey85", lty = "dotted")

# Add boundary lines for classification regions
abline(v = 2.5, lty = 2, col = "grey40", lwd = 1.5)
```

```

abline(v = 5.0, lty = 2, col = "grey40", lwd = 1.5)
abline(h = 0.75, lty = 2, col = "grey40", lwd = 1.5)
abline(h = 1.75, lty = 2, col = "grey40", lwd = 1.5)

# Legend
legend("topleft",
      legend = names(species_colors),
      col     = species_colors,
      pch     = species_pch,
      bty     = "n", cex = 1.1, title = "Species (Classified)")

# Figure 2: Boxplot of Sepal Length per species
boxplot(Sepal.Length ~ Species, data = iris,
      col = c("firebrick", "steelblue", "forestgreen"),
      border = "grey30",
      xlab = "Species",
      ylab = "Sepal Length (cm)",
      main = "Figure 2: Sepal Length by Species",
      cex.axis = 1.1, cex.lab = 1.2, cex.main = 1.3,
      las = 1)
grid(nx = NA, ny = NULL, col = "grey85", lty = "dotted")

dev.off()
cat("PDF saved as iris_plots.pdf\n")

```

The two figures are stacked vertically using `par(mfrow = c(2,1))` inside a single PDF device. The classification boundary lines are added with `abline()` to make the three species regions visually clear.

EXTRA PRACTICE ASSIGNMENTS — Experiment 2

Assignment 1: Create a weight vs height plot distinguishing males and females with legend. Repeat using `ggplot2`.

Answer:

```

weight <- c(55,85,75,42,93,63,58,75,89,67)
height <- c(161,185,174,154,188,178,170,167,181,178)
sex    <- c("F","M","M","F","M","M","F","M","M","F")

# Base R
plot(weight, height,
     col = ifelse(sex == "M", "steelblue", "firebrick"),
     pch = ifelse(sex == "M", 16, 17),

```

```

    xlab = "Weight (kg)", ylab = "Height (cm)",
    main = "Weight vs Height by Sex", cex = 1.4, las = 1)
legend("topleft",
      legend = c("Male", "Female"),
      col     = c("steelblue", "firebrick"),
      pch     = c(16, 17), bty = "n")

# ggplot2
library(ggplot2)
df <- data.frame(weight, height, sex)
ggplot(df, aes(x=weight, y=height, color=sex, shape=sex)) +
  geom_point(size=4) +
  scale_color_manual(values=c("F"="firebrick", "M"="steelblue")) +
  scale_shape_manual(values=c("F"=17, "M"=16)) +
  labs(title="Weight vs Height by Sex",
       x="Weight (kg)", y="Height (cm)",
       color="Sex", shape="Sex") +
  theme_minimal()

```

Assignment 2: Using stackloss — explore relationships with scatter-plots and use pairs() for all pairwise plots.

Answer:

```

data(stackloss)

# (a) Individual scatterplots
par(mfrow = c(1, 3))
plot(stackloss$Air.Flow, stackloss$stack.loss,
     xlab="Air Flow", ylab="Stack Loss", main="Air Flow vs Stack Loss", pch=16)
plot(stackloss$Water.Temp, stackloss$stack.loss,
     xlab="Water Temp", ylab="Stack Loss", main="Water Temp vs Stack Loss", pch=16)
plot(stackloss$Acid.Conc., stackloss$stack.loss,
     xlab="Acid Conc.", ylab="Stack Loss", main="Acid Conc vs Stack Loss", pch=16)

# (b) All pairwise scatterplots
pairs(stackloss,
      main = "All Pairwise Scatterplots - stackloss",
      pch = 16, col = "steelblue")

```

Air.Flow and Water.Temp show strong positive linear relationships with stack.loss. Acid.Conc. shows a weaker, possibly non-linear relationship.

Assignment 3: Boxplot of mpg split by cyl and scatterplot with regression line for mtcars, stacked vertically.

Answer:

```
data(mtcars)
par(mfrow = c(2, 1), mar = c(4, 4, 3, 1))

# Boxplot: mpg by cyl
boxplot(mpg ~ cyl, data = mtcars,
        col = c("steelblue", "gold", "firebrick"),
        xlab = "Cylinders", ylab = "MPG",
        main = "MPG by Number of Cylinders", las = 1)

# Scatterplot with regression line
plot(mtcars$cyl, mtcars$mpg,
     xlab = "Cylinders", ylab = "MPG",
     main = "MPG vs Cylinders with Regression",
     pch = 16, col = "steelblue", las = 1)
abline(lm(mpg ~ cyl, data = mtcars), col = "firebrick", lwd = 2)
```

Assignment 4: penguins dataset — Flipper length boxplots by species and sex using both base R and ggplot2, side by side.

Answer:

```
library(palmerpenguins)
library(ggplot2)
data(penguins)
penguins <- na.omit(penguins)

par(mfrow = c(1, 2))

# Base R: boxplot with species-sex formula
boxplot(flipper_length_mm ~ species + sex, data = penguins,
        col = c("steelblue", "firebrick", "forestgreen",
                "lightblue", "pink", "lightgreen"),
        las = 2, ylab = "Flipper Length (mm)",
        main = "Base R: Flipper Length")

# ggplot2
p <- ggplot(penguins, aes(x=species, y=flipper_length_mm, fill=sex)) +
  geom_boxplot() +
  scale_fill_manual(values=c("male"="steelblue", "female"="firebrick")) +
  labs(title="ggplot2: Flipper Length by Species and Sex",
       x="Species", y="Flipper Length (mm)") +
```

```
theme_minimal()
print(p)
```

Assignment 5: Open a 9×4.5 inch device, set layout (Fig1 spans 2 rows on left; Fig2 top-right; Fig3 bottom-right). Load survey from MASS package and plot.

Answer:

```
library(MASS)
data(survey)
survey <- na.omit(survey[, c("Wr.Hnd", "NW.Hnd", "Height", "Smoke", "Pulse", "Exer")])

dev.new(width=9, height=4.5)
layout(matrix(c(1,2,1,3), nrow=2, ncol=2))
par(mar = c(4, 4, 2, 1)) # bottom, left, top, right

# Fig 1 (left, spans 2 rows): Wr.Hnd vs NW.Hnd
plot(survey$Wr.Hnd, survey$NW.Hnd,
     xlab = "Writing Hand Span (cm)",
     ylab = "Non-writing Hand Span (cm)",
     pch = 16, col = "steelblue")

# Fig 2 (top right): Height by Smoke
boxplot(Height ~ Smoke, data = survey, horizontal = TRUE,
        xlab = "Height (cm)", ylab = "Smoking Status", las = 1)

# Fig 3 (bottom right): Pulse by Exercise
boxplot(Pulse ~ Exer, data = survey, horizontal = TRUE,
        xlab = "Pulse (bpm)", ylab = "Exercise Level", las = 1)
```

Assignment 6: Using InsectSprays — count insect deaths per spray using aggregate() and create notched boxplots.

Answer:

```
data(InsectSprays)

# Count deaths per spray using aggregate
insect_counts <- aggregate(count ~ spray, data = InsectSprays, FUN = sum)
print(insect_counts)

# Notched boxplots
boxplot(count ~ spray, data = InsectSprays,
```



```
notch = TRUE,  
col    = rainbow(6),  
xlab   = "Insect Spray Type",  
ylab   = "Insect Count",  
main   = "Insect Deaths by Spray Type (Notched Boxplot)",  
las    = 1)
```

THEORY QUESTIONS — Experiment 2

Q1. Discuss the importance of data visualization in communicating insights.

Answer:

Data visualization converts raw numbers into graphical representations, making patterns, trends, and outliers immediately apparent. Its importance:

- **Rapid comprehension:** Stakeholders can understand complex datasets quickly through charts and graphs
- **Exploratory Data Analysis (EDA):** Helps discover hidden structures, clusters, and relationships before formal analysis
- **Hypothesis generation:** Patterns in plots guide the formulation of testable hypotheses
- **Effective communication:** A well-designed plot communicates findings to non-technical audiences far better than tables of numbers
- **Outlier detection:** Visual inspection quickly reveals anomalies that summary statistics might miss

Without visualization, even a skilled statistician can miss important structure in data.

Q2. Write about a scatter plot with an example in R. Explain its importance.

Answer:

A **scatter plot** displays the relationship between two continuous variables. Each point represents one observation. It reveals correlation, clusters, outliers, and linearity (or non-linearity).

Importance: Shows direction and strength of association, identifies outliers, and visually detects whether a linear model is appropriate.

```
plot(iris$Sepal.Length, iris$Petal.Length,
     xlab = "Sepal Length", ylab = "Petal Length",
     main = "Sepal vs Petal Length",
     pch = 16, col = "steelblue")
```

Q3. Write a short note on pairs() function in R.

Answer:

`pairs()` creates a **scatterplot matrix** — all pairwise scatter plots for a set of variables arranged in a grid. It is extremely useful for identifying correlations between multiple variables simultaneously without creating individual plots.

```
pairs(iris[,1:4],
      col = c("red", "blue", "green")[iris$Species],
      pch = 16,
      main = "Pairwise Scatterplot Matrix - Iris")
```

For a dataset with 4 variables, `pairs()` generates a 4×4 grid = 16 plots (the diagonal is blank or can show variable names). Each off-diagonal cell shows the scatter of one variable against another.

Q4. Write about a boxplot with an example in R. Explain its importance.

Answer:

A **boxplot** (box-and-whisker plot) shows the **five-number summary**: minimum, Q1, median, Q3, and maximum, along with outliers.

- The **box** spans the IQR (Q1 to Q3)
- The **line** inside the box shows the median
- **Whiskers** extend to $1.5 \times \text{IQR}$ beyond the box
- Points beyond whiskers are plotted individually as potential outliers

Importance: Compares distributions across groups, shows spread and skewness, and highlights outliers.

```
boxplot(Sepal.Length ~ Species, data = iris,
        col = c("red", "green", "blue"),
        main = "Sepal Length by Species",
        xlab = "Species", ylab = "Sepal Length (cm)")
```

Q5. Describe the difference between `mfrow()`, `mfcol()` and `layout()` in R.

Answer:

Function	Behaviour
<code>par(mfrow=c(r,c))</code>	Divides device into $r \times c$ grid; fills panels row-by-row
<code>par(mfcol=c(r,c))</code>	Same grid but fills panels column-by-column
<code>layout(matrix)</code>	Most flexible — accepts a matrix specifying which panel occupies which cell; allows panels of different sizes and merged cells

Example of `layout()`:

```
# Panel 1 spans the left column (2 rows); panels 2 and 3 on the right
layout(matrix(c(1,2,1,3), nrow=2, ncol=2))
```

Q6. Differentiate between base R plotting and ggplot2.

Answer:

Aspect	Base R	ggplot2
Paradigm	Procedural — build plot step by step	Grammar of Graphics — data + aesthetics + geoms + layers
Syntax	<code>plot()</code> , then <code>points()</code> , <code>lines()</code> , <code>legend()</code> etc.	<code>ggplot()</code> + <code>geom_*()</code> + <code>labs()</code> + <code>theme()</code>
Customization	Flexible but verbose for complex plots	Concise; grouping, faceting, and theming are simple
Output quality	Good for quick EDA	More polished, publication-ready output
Speed	Faster for large datasets	Slightly slower
Learning curve	Shallower	Steeper initially

Q7. Explain the basic structure of a ggplot2 plot.

Answer:

A ggplot2 plot is built from **five key layers**:

```
ggplot(data = iris,                                # 1. DATA
       aes(x = Petal.Length,                        # 2. AESTHETICS (aes)
           y = Petal.Width,
           color = Species)) +
  geom_point(size = 3) +                            # 3. GEOMETRY (geom_*)
  labs(title = "Iris Petal Dimensions",             # 4. LABELS
       x = "Petal Length",
       y = "Petal Width") +
  theme_minimal()                                   # 5. THEME
```

Layer	Purpose
Data	The data frame being plotted
Aesthetics (aes)	Maps variables to visual properties (x, y, color, shape, size)
Geometry (geom_*)	Defines the type of plot (points, lines, bars, boxes)
Labels (labs)	Title, axis labels, legend titles
Theme	Controls overall appearance (fonts, background, gridlines)

EXPERIMENT 3: Descriptive Statistics and Correlation in R

MAIN PROBLEM STATEMENT

Dataset: Indian Food Nutritional Values from Kaggle.

Question (a): Create a subset of all soups named “soups” using `grep()`.

Answer:

```
food <- read.csv("indian_food_nutritional_values.csv",
                 stringsAsFactors = FALSE)

soups <- food[grep("soup", food$name, ignore.case = TRUE), ]
cat("Number of soup entries:", nrow(soups), "\n")
print(soups$name)
```

Question (b): Find the average macronutrient values (carbohydrates, proteins, fats) in soups using `sapply()`.

Answer:

```
# Assuming columns: carbohydrates, proteins, fat (verify with names(soups))
macro_cols <- c("carbohydrates", "proteins", "fat")
avg_macros <- sapply(soups[, macro_cols], mean, na.rm = TRUE)
cat("Average macronutrients in soups (per 100g):\n")
print(round(avg_macros, 2))
```

Use `names(soups)` to verify exact column names in your dataset.

Question (c): What is the median fibre count of soups?

Answer:

```
median_fibre <- median(soups$fibre, na.rm = TRUE)
cat("Median fibre count of soups:", median_fibre, "g\n")
```

Question (d): How many mg of iron do most soups have?

Answer:

```
# Most common iron value (mode approach)
iron_table <- table(soups$iron)
most_common_iron <- as.numeric(names(which.max(iron_table)))
cat("Most common iron content in soups:", most_common_iron, "mg\n")

# Summary of iron in soups
cat("Iron summary:\n")
print(summary(soups$iron))
```

Question (e): Create a subset of protein-rich dishes (more than 15g/100g) using `which()`.

Answer:

```
protein_rich_idx <- which(food$proteins > 15)
protein_rich     <- food[protein_rich_idx, ]
cat("Number of protein-rich dishes (>15g):", nrow(protein_rich), "\n")
print(protein_rich$name)
```

Question (f): Compare the variance in protein of all dishes vs protein-rich dishes.

Answer:

```
var_all <- var(food$proteins, na.rm = TRUE)
var_pr  <- var(protein_rich$proteins, na.rm = TRUE)
sd_all  <- sd(food$proteins, na.rm = TRUE)
sd_pr   <- sd(protein_rich$proteins, na.rm = TRUE)

cat("Variance - All dishes:    ", round(var_all, 4), "\n")
cat("Variance - Protein-rich:  ", round(var_pr, 4), "\n")
cat("Std Dev - All dishes:     ", round(sd_all, 4), "\n")
cat("Std Dev - Protein-rich:   ", round(sd_pr, 4), "\n")
```

The protein-rich subset has **lower variance** because all values are already above 15g — the range is narrower than the full dataset which includes zero/low-protein items.

Question (g): Compare the skewness of calories and sodium in protein-rich dishes.

Answer:

```
library(moments) # install.packages("moments") if needed

skew_cal <- skewness(protein_rich$calories, na.rm = TRUE)
skew_sod <- skewness(protein_rich$sodium,   na.rm = TRUE)

cat("Skewness of Calories (protein-rich):", round(skew_cal, 4), "\n")
cat("Skewness of Sodium (protein-rich):", round(skew_sod, 4), "\n")

# Interpretation function
interpret_skew <- function(s, var_name) {
  if (s > 0.5)      cat(var_name, "is positively skewed (right tail)\n")
  else if (s < -0.5) cat(var_name, "is negatively skewed (left tail)\n")
}
```

```

    else                cat(var_name, "is approximately symmetric\n")
  }
interpret_skew(skew_cal, "Calories")
interpret_skew(skew_sod, "Sodium")

```

Question (h): What correlation exists between protein and calcium content in protein-rich dishes?

Answer:

```

cor_prot_cal <- cor(protein_rich$proteins, protein_rich$calcium,
                    use = "complete.obs", method = "pearson")
cat("Pearson correlation (protein vs calcium):", round(cor_prot_cal, 4), "\n")

# Significance test
cor_test <- cor.test(protein_rich$proteins, protein_rich$calcium)
print(cor_test)

# Interpretation
if (abs(cor_prot_cal) > 0.7)    cat("Strong correlation\n")
else if (abs(cor_prot_cal) > 0.4) cat("Moderate correlation\n")
else                          cat("Weak correlation\n")

```

Question (i): Use appropriate functions to summarize both subsets.

Answer:

```

cat("=== Summary of SOUPS subset ===\n")
summary(soups[, c("carbohydrates", "proteins", "fat", "fibre", "calories", "sodium")])

cat("\n=== Summary of PROTEIN-RICH dishes subset ===\n")
summary(protein_rich[, c("carbohydrates", "proteins", "fat", "fibre", "calories", "sodium")])

```

EXTRA PRACTICE ASSIGNMENTS — Experiment 3

Assignment 1: InsectSprays — mode of insect counts, total by spray, and % with 5 bugs.

Answer:

```
data(InsectSprays)
```

```

# (a) Mode of insect counts
count_table <- table(InsectSprays$count)
mode_val <- as.numeric(names(count_table[count_table == max(count_table)]))
cat("Mode(s) of insect counts:", mode_val, "\n")

# (b) Total count per spray type
total_by_spray <- tapply(InsectSprays$count, InsectSprays$spray, sum)
print(total_by_spray)

# (c) % units with >= 5 bugs per spray (rounded)
pct_5plus <- tapply(InsectSprays$count, InsectSprays$spray,
  function(x) round(mean(x >= 5) * 100))
cat("% agricultural units with >=5 bugs per spray:\n")
print(pct_5plus)

```

Assignment 2: quakes — IQR of depths, five-number summary for deep quakes, depth categories, and statistics by category.

Answer:

```

data(quakes)

# (a) IQR of depths
cat("IQR of depths:", IQR(quakes$depth), "km\n")

# (b) Five-number summary of magnitudes at depth >= 400 km
deep_quakes <- quakes[quakes$depth >= 400, ]
cat("Five-number summary (depth >= 400 km):\n")
print(fivenum(deep_quakes$mag))

# (c) Cut depth into 4 evenly spaced categories
depthcat <- cut(quakes$depth,
  breaks = c(40, 200, 360, 520, 680),
  right = FALSE,
  labels = c("[40,200)", "[200,360)", "[360,520)", "[520,680)"))
levels(depthcat)

# (d) Mean and SD of magnitudes per depth category
mean_by_cat <- tapply(quakes$mag, depthcat, mean)
sd_by_cat <- tapply(quakes$mag, depthcat, sd)
cat("Mean magnitude by depth category:\n"); print(round(mean_by_cat, 3))
cat("SD magnitude by depth category:\n"); print(round(sd_by_cat, 3))

# (e) 0.8 quantile of magnitudes by depthcat
q80 <- tapply(quakes$mag, depthcat, quantile, probs = 0.8)

```



```
cat("0.8th quantile by depth category:\n"); print(q80)
```

Assignment 3: anorexia dataset — weight change analysis.

Answer:

```
library(MASS)
data(anorexia)
anorexia$weight_change <- anorexia$Postwt - anorexia$Prewt

# (a) Mean weight gain/loss
cat("Mean weight change:", round(mean(anorexia$weight_change), 3), "lbs\n")

# (b) Weight change by treatment
tapply(anorexia$weight_change, anorexia$Treat, mean)

# (c) Skewness of pre-treatment weights
library(moments)
cat("Skewness of Prewt:", round(skewness(anorexia$Prewt), 4), "\n")

# (d) Variability in post-treatment weights by group
tapply(anorexia$Postwt, anorexia$Treat, sd)

# (e) Correlation: pre-treatment weight vs weight change
cat("Correlation (Prewt vs change):",
    round(cor(anorexia$Prewt, anorexia$weight_change), 4), "\n")
```

SOLVE (Manual Calculations) — Experiment 3

Solve 1: Compute Pearson's r and Spearman's ρ for IQ vs Exam Marks.

Data: IQ = {110, 100, 140, 120, 80, 90}, Exam = {70, 60, 80, 60, 10, 20}

Manual Calculation:

- Mean IQ: $\bar{x} = (110+100+140+120+80+90)/6 = 640/6 = \mathbf{106.67}$
- Mean Exam: $\bar{y} = (70+60+80+60+10+20)/6 = 300/6 = \mathbf{50}$
- $\text{Cov}(x,y) = \Sigma(x_i - \bar{x})(y_i - \bar{y})/(n-1) = 2700/5 = \mathbf{540}$
- $\text{SD}(x) = \sqrt{(2333.54/5)} = \sqrt{466.71} = \mathbf{21.60}$
- $\text{SD}(y) = \sqrt{(4000/5)} = \sqrt{800} = \mathbf{28.28}$
- $\text{Pearson } r = 540 / (21.60 \times 28.28) = \mathbf{0.884}$

R Verification:

```
IQ <- c(110, 100, 140, 120, 80, 90)
Exam <- c(70, 60, 80, 60, 10, 20)

pearson_r <- cor(IQ, Exam, method = "pearson")
spearman_r <- cor(IQ, Exam, method = "spearman")
cat("Pearson r :", round(pearson_r, 4), "\n")
cat("Spearman :", round(spearman_r, 4), "\n")

# Using cov()
r_via_cov <- cov(IQ, Exam) / (sd(IQ) * sd(Exam))
cat("Pearson via cov():", round(r_via_cov, 4), "\n")

Pearson r 0.884 (strong positive). Spearman 0.829 (strong
positive). The rank-based Spearman is slightly lower due to ties in
exam marks at 60.
```

Solve 2: Pearson's r and Spearman's for Mathematics vs Statistics marks (10 students).

Data: Math = {75,30,60,80,53,35,14,40,38,48}, Stat = {85,45,54,91,58,63,35,43,42,44}

Answer:

```
Math <- c(75, 30, 60, 80, 53, 35, 14, 40, 38, 48)
Stat <- c(85, 45, 54, 91, 58, 63, 35, 43, 42, 44)

pearson_r <- cor(Math, Stat, method = "pearson")
spearman_r <- cor(Math, Stat, method = "spearman")
cat("Pearson r (Math vs Stat):", round(pearson_r, 4), "\n")
cat("Spearman (Math vs Stat):", round(spearman_r, 4), "\n")

# Via cov()
r_via_cov <- cov(Math, Stat) / (sd(Math) * sd(Stat))
cat("Pearson via cov():", round(r_via_cov, 4), "\n")

Pearson r 0.964 — very strong positive linear correlation between
Maths and Statistics marks.
```

THEORY QUESTIONS — Experiment 3

Q1. Define measures of central tendency, list any three measures and demonstrate R implementation.

Answer:

Central tendency describes the center/typical value of a distribution. Three main measures:

- **Mean:** Arithmetic average. Sensitive to outliers.
- **Median:** Middle value when sorted. Robust to outliers.
- **Mode:** Most frequently occurring value.

```
x <- c(2, 4, 4, 6, 8, 10)
mean(x)                # 5.67
median(x)              # 5
names(which.max(table(x))) # "4" (mode)
```

Q2. Define dispersion, list any three measures and demonstrate R implementation.

Answer:

Dispersion describes the spread of data around the center. Three measures:

- **Range:** $\max - \min$
- **Variance:** Average squared deviation from the mean
- **Standard Deviation:** Square root of variance

```
x <- c(2, 4, 4, 6, 8, 10)
diff(range(x))    # Range = 8
var(x)            # Variance 8.267
sd(x)             # SD 2.875
```

Q3. Differentiate between quartiles, IQR, and quantiles with R functions for computation.

Answer:

- **Quartiles:** Divide sorted data into 4 equal parts. Q1 = 25th percentile, Q2 = median (50th), Q3 = 75th percentile.
- **IQR** (Interquartile Range): $= Q3 - Q1$, measuring the middle 50% spread. Robust measure of dispersion.
- **Quantiles:** Generalize quartiles to any proportion (e.g., 0.1, 0.9 = 10th and 90th percentiles).

```
x <- quakes$depth
quantile(x, 0.25)      # Q1
quantile(x, 0.75)      # Q3
```

```
IQR(x)                                # Q3 - Q1
quantile(x, c(0.1, 0.9))              # 10th and 90th percentile
```

Q4. Define skewness, explain positive and negative skewness, and state conditions for a distribution to be skewed.

Answer:

Skewness measures the asymmetry of a distribution around its mean.

- **Positive skewness (right skew):** Tail extends to the right; mean > median > mode. Most values cluster on the left.
- **Negative skewness (left skew):** Tail extends to the left; mean < median < mode. Most values cluster on the right.
- **Symmetric:** Skewness = 0; mean = median = mode (e.g., normal distribution).

Conditions: - $|\text{skewness}| > 1 \rightarrow$ highly skewed - $0.5 \leq |\text{skewness}| \leq 1 \rightarrow$ moderately skewed - $|\text{skewness}| < 0.5 \rightarrow$ approximately symmetric

Q5. Describe kurtosis and how it is measured using the ‘moments’ package in R.

Answer:

Kurtosis measures the “tailedness” or peakedness of a distribution.

- **Mesokurtic** (excess kurtosis = 0): Normal distribution
- **Leptokurtic** (excess kurtosis > 0): Heavy tails, sharp peak — more extreme values
- **Platykurtic** (excess kurtosis < 0): Light tails, flat peak — fewer extreme values

```
library(moments)
kurtosis(rnorm(1000))              # 3 (excess 0, normal)
kurtosis(c(1,1,1,10,10,10,10,100)) # Higher kurtosis (heavy tail)
```

Note: The `kurtosis()` function from the `moments` package returns the **raw kurtosis** (normal = 3). Excess kurtosis = kurtosis - 3.

Q6. Explain the `sapply()` function for descriptive statistics, including syntax.

Answer:

`sapply(X, FUN)` applies a function `FUN` to each element of `X` (list or data frame columns) and **simplifies** the result to a vector or matrix if possible.

Syntax: `sapply(X, FUN, ...)`

```
sapply(iris[,1:4], mean)    # mean of each of the 4 numeric columns

sapply(iris[,1:4], sd)      # SD of each column

# Multiple statistics at once (returns a matrix)
sapply(iris[,1:4], function(x) c(mean=mean(x), sd=sd(x)))
```

Q7. Compare `summary()` and `fivenum()` in R, highlighting differences in quartile calculation.

Answer:

Feature	<code>summary()</code>	<code>fivenum()</code>
Output	Min, Q1, Median, Mean , Q3, Max	Min, Lower-hinge, Median, Upper-hinge, Max
Algorithm	Type 7 quantile (default R)	Tukey's method (Type 2)
Handles factors	Yes	No (numeric only)
Mean included	Yes	No
Quartile values	May differ for small samples	Uses Tukey's hinges

```
x <- quakes$depth
summary(x)    # includes mean
fivenum(x)    # Tukey's five-number summary (no mean)
```

Q8. Describe the `aggregate()` function for descriptive statistics by group, including syntax for multiple grouping variables.

Answer:

`aggregate(formula, data, FUN)` splits data by grouping variables and applies a function to each group.

Syntax: `aggregate(response ~ group, data = df, FUN = function_name)`

```
# Single grouping variable
aggregate(Sepal.Length ~ Species, data = iris, FUN = mean)

# Multiple response variables
```

```
aggregate(cbind(Sepal.Length, Petal.Length) ~ Species, data = iris, FUN = sd)

# Multiple grouping variables
aggregate(Length ~ AirBags + Man.trans.avail, data = Cars93, FUN = mean)
```

Q9. Define correlation in bivariate distributions, distinguishing positive, negative, and perfect correlation.

Answer:

Correlation measures the strength and direction of a **linear relationship** between two variables, ranging from -1 to $+1$.

Type	r value	Meaning
Perfect positive	$r = +1$	Both increase/decrease together exactly
Strong positive	$r > 0.7$	Strong tendency to increase together
No correlation	$r = 0$	No linear relationship
Strong negative	$r < -0.7$	As one increases, other decreases strongly
Perfect negative	$r = -1$	Exact inverse relationship

Correlation does not imply causation.

Q10. Discuss Pearson and Spearman correlation coefficients.

Answer:

Pearson's r: - Measures **linear** correlation between two continuous variables - Assumes **normality** and is sensitive to outliers - Formula: $r = \text{cov}(x,y) / (\text{sd}(x) \times \text{sd}(y))$ - Range: $[-1, 1]$

Spearman's : - **Rank-based** — replaces values with their ranks, then computes Pearson's r on ranks - **Non-parametric**, robust to outliers and non-linear monotonic relationships - Formula: $r = 1 - (6\sum d_i^2) / (n(n^2-1))$, where d_i = rank difference - Suitable when data is ordinal or non-normal

```
cor(IQ, Exam, method = "pearson")    # Pearson r
cor(IQ, Exam, method = "spearman")   # Spearman
```

EXPERIMENT 4: Hypothesis Testing in R

MAIN PROBLEM STATEMENT

Question (a): `PlantGrowth` — Set up hypotheses, check variance equality, conduct t-test, and conclude.

Hypotheses: - H_0 : $\mu_{ctrl} = \mu_{trt}$ (control yield = treatment yield) - H_a : $\mu_{ctrl} < \mu_{trt}$ (one-tailed left)

Answer:

```
data(PlantGrowth)

# Extract groups
ctrl <- PlantGrowth$weight[PlantGrowth$group == "ctrl"]
trt1 <- PlantGrowth$weight[PlantGrowth$group == "trt1"]
trt2 <- PlantGrowth$weight[PlantGrowth$group == "trt2"]

# Step 1: F-test for equality of variances
var.test(ctrl, trt1) # p > 0.05 → use pooled (var.equal = TRUE)
var.test(ctrl, trt2) # check similarly

# Step 2: t-test ctrl vs trt1 (one-tailed: ctrl < trt1)
t.test(ctrl, trt1,
       alternative = "less",
       var.equal = TRUE,
       conf.level = 0.95)

# Step 3: t-test ctrl vs trt2
t.test(ctrl, trt2,
       alternative = "less",
       var.equal = TRUE,
       conf.level = 0.95)
```

ctrl vs trt1: p-value > 0.05 → Fail to reject H_0 . Control yield is NOT significantly less than trt1. **ctrl vs trt2:** p-value < 0.05 → Reject H_0 . Control yield IS significantly less than trt2, suggesting

trt2 is effective. If var.test gives $p < 0.05$, use `var.equal=FALSE` (Welch's t-test) for that pair.

Question (b): Anti-malarial quinine campaign — Chi-square test (manual + R).

Contingency Table:

Treatment	Fever	No Fever	Total
Quinine	20	792	812
No Quinine	220	2216	2436
Total	240	3008	3248

Hypotheses: - H_0 : Quinine has no effect on preventing fever (independent) -
 H_a : Quinine is useful in preventing fever (associated)

Manual Calculation:

Expected frequency formula: $E_{ij} = (\text{Row total} \times \text{Column total}) / \text{Grand total}$

- $E(\text{Quinine, Fever}) = (812 \times 240) / 3248 = \mathbf{60.0}$
- $E(\text{Quinine, No Fever}) = (812 \times 3008) / 3248 = \mathbf{752.0}$
- $E(\text{No Quinine, Fever}) = (2436 \times 240) / 3248 = \mathbf{180.0}$
- $E(\text{No Quinine, No Fever}) = (2436 \times 3008) / 3248 = \mathbf{2256.0}$

$$\chi^2 = \sum (O - E)^2 / E = (20-60)^2/60 + (792-752)^2/752 + (220-180)^2/180 + (2216-2256)^2/2256 = 1600/60 + 1600/752 + 1600/180 + 1600/2256 = \mathbf{26.667} + \mathbf{2.128} + \mathbf{8.889} + \mathbf{0.709} = \mathbf{38.39}$$

$$df = (2-1)(2-1) = 1. \text{ Critical } \chi^2(1, \alpha=0.05) = 3.841$$

Since $38.39 \gg 3.841$, reject H_0 . **Quinine IS effective** in preventing malaria.

R Code:

```

malaria_table <- matrix(c(20, 792, 220, 2216),
                        nrow = 2, byrow = TRUE,
                        dimnames = list(
                          Treatment = c("Quinine", "No Quinine"),
                          Outcome   = c("Fever", "No Fever")))

print(malaria_table)

chi_result <- chisq.test(malaria_table, correct = FALSE)
print(chi_result)
cat("Chi-squared statistic:", round(chi_result$statistic, 4), "\n")
cat("p-value:", chi_result$p.value, "\n")

```



```
cat("Expected frequencies:\n")
print(chi_result$expected)
```

² 38.39, p-value « 0.001. Strong evidence that quinine is effective in preventing malarial fever.

EXTRA PRACTICE ASSIGNMENTS — Experiment 4

Assignment 1: quakes — One-sample t-test. Test if true mean magnitude > 4.3 at $\alpha = 0.01$.

Hypotheses: $H_0: \mu = 4.3$ vs $H_a: \mu > 4.3$ (one-tailed right)

Answer:

```
data(quakes)
t.test(quakes$mag,
       mu = 4.3,
       alternative = "greater",
       conf.level = 0.99) # = 0.01
```

If p-value < 0.01, reject H_0 : true mean magnitude is significantly greater than 4.3.

Assignment 2: Cat weights — One-sample t-test. $n=73$, $\bar{x}=3.97$, $s=2.21$, $\mu_0=3.5$, $\alpha=0.05$.

Hypotheses: $H_0: \mu = 3.5$ vs $H_a: \mu \neq 3.5$ (two-tailed)

Manual Calculation:

$t = (\bar{x} - \mu_0) / (s/\sqrt{n}) = (3.97 - 3.5) / (2.21/\sqrt{73}) = 0.47 / 0.2587 = 1.816$

$df = 72$. Critical $t(72, \alpha/2=0.025) = 1.993$.

Since $|t| = 1.816 < 1.993 \rightarrow$ **Fail to reject H_0** .

Answer:

```
n <- 73; xbar <- 3.97; s <- 2.21; mu0 <- 3.5
t_stat <- (xbar - mu0) / (s / sqrt(n))
p_val <- 2 * pt(-abs(t_stat), df = n-1)
cat("t-statistic:", round(t_stat, 4), "\n")
cat("p-value: ", round(p_val, 4), "\n")
cat("Conclusion: ", ifelse(p_val < 0.05, "Reject H0", "Fail to reject H0"), "\n")
```

Assignment 3: anorexia — Paired t-test overall and by treatment group.

Hypotheses: $H: d = 0$ vs $H: d \neq 0$

Answer:

```
library(MASS)
data(anorexia)

# (a) Overall paired t-test for all participants
t.test(anorexia$Postwt, anorexia$Prewt,
       paired = TRUE,
       alternative = "two.sided",
       conf.level = 0.95)

# (b) Separate t-tests per treatment group
for (grp in unique(anorexia$Treat)) {
  sub <- anorexia[anorexia$Treat == grp, ]
  cat("\n=== Treatment:", grp, "===\n")
  print(t.test(sub$Postwt, sub$Prewt, paired=TRUE, alternative="two.sided"))
}
```

Assignment 4: iris — Diagnostic checks and one-way ANOVA.

Answer:

```
data(iris)
library(car)

# (a) Check equality of variances for all 4 measurements
vars <- c("Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width")
for (v in vars) {
  lt <- leveneTest(iris[[v]] ~ iris$Species)
  cat(v, "- Levene p:", round(lt$"Pr(>F)"[1], 4),
      ifelse(lt$"Pr(>F)"[1] > 0.05, "(suitable for ANOVA)", "(NOT suitable)"), "\n")
}

# (b) One-way ANOVA for suitable variables (e.g., Sepal.Width)
aov_sw <- aov(Sepal.Width ~ Species, data = iris)
summary(aov_sw)

# Post-hoc Tukey test
TukeyHSD(aov_sw)
```

Assignment 5: Cars93 — Mean length by AirBags & Man.trans.avail, one-way and two-way ANOVA.

Answer:

```
library(MASS)
data(Cars93)

# (a) Mean length by AirBags and Man.trans.avail
aggregate(Length ~ AirBags + Man.trans.avail, data = Cars93, FUN = mean)

# (b) One-way ANOVA for Length by AirBags
aov_one <- aov(Length ~ AirBags, data = Cars93)
summary(aov_one)

# (c) Full two-way ANOVA with interaction
aov_two <- aov(Length ~ AirBags * Man.trans.avail, data = Cars93)
summary(aov_two)
# If interaction  $p > 0.05$ : not significant; interpret main effects only
```

Assignment 6: mtcars — Simple Linear Regression of mpg on wt.

Answer:

```
data(mtcars)

# (a) Plot mpg vs wt
plot(mtcars$wt, mtcars$mpg,
     xlab = "Weight (1000 lbs)", ylab = "Miles per Gallon",
     main = "MPG vs Vehicle Weight",
     pch = 16, col = "steelblue", las = 1)

# (b) Fit SLR model and add fitted line
model <- lm(mpg ~ wt, data = mtcars)
abline(model, col = "firebrick", lwd = 2)

# (c) Summary and equation
summary(model)
# Equation:  $mpg = 37.285 - 5.344 \times wt$ 
# Slope = -5.344: each 1000 lb increase in weight  $\rightarrow$  mpg decreases by 5.344

# (d) Predict for 6000 lbs (wt = 6)
new_data <- data.frame(wt = 6)
predict(model, newdata = new_data, interval = "confidence", level = 0.95)
```

Regression equation: $MPG = 37.285 - 5.344 \times \text{Weight}$. The

slope is statistically significant ($p \ll 0.001$). For a 6000 lb car: predicted MPG $37.285 - 5.344 \times 6$ **5.22 mpg**.

Assignment 7: survey — Height on writing handspan regression; predict for handspans 12, 15.2, 17, 19.9 cm.

Answer:

```
library(MASS)
data(survey)
survey_clean <- na.omit(survey[, c("Height", "Wr.Hnd")])

survfit <- lm(Height ~ Wr.Hnd, data = survey_clean)
summary(survfit)

new_hs <- data.frame(Wr.Hnd = c(12, 15.2, 17, 19.9))
predict(survfit, newdata = new_hs,
        interval = "confidence", level = 0.99)
```

Assignment 8: BreastCancer — Logistic regression with cell thickness, size, and shape.

Answer:

```
library(mlbench)
data(BreastCancer)
bc <- na.omit(BreastCancer)

# Convert to numeric
bc$Cl.thickness <- as.numeric(as.character(bc$Cl.thickness))
bc$Cell.size <- as.numeric(as.character(bc$Cell.size))
bc$Cell.shape <- as.numeric(as.character(bc$Cell.shape))
bc$Class_bin <- ifelse(bc$Class == "malignant", 1, 0)

# Fit logistic regression
log_model <- glm(Class_bin ~ Cl.thickness + Cell.size + Cell.shape,
                 data = bc,
                 family = binomial(link = "logit"))
summary(log_model)

# Odds ratios
exp(coef(log_model))

# Predict and evaluate accuracy
```

```
bc$predicted <- ifelse(predict(log_model, type="response") > 0.5, 1, 0)
conf_matrix <- table(Actual=bc$Class_bin, Predicted=bc$predicted)
print(conf_matrix)
accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
cat("Accuracy:", round(accuracy*100, 2), "%\n")
```

THEORY QUESTIONS — Experiment 4

Q1. Differentiate between population and sample, and parameter and statistic.

Answer:

- **Population:** The complete set of all individuals/items under study (e.g., all adults in India).
- **Sample:** A subset drawn from the population for analysis (e.g., 1000 adults surveyed).
- **Parameter:** A numerical measure describing a **population** characteristic (e.g., μ = population mean, σ = population SD).
- **Statistic:** A numerical measure computed from a **sample** (e.g., $\bar{x}$ = sample mean, s = sample SD).

Since populations are usually too large to measure completely, sample statistics are used to **estimate** population parameters.

Q2. Define hypothesis testing components: null/alternative hypotheses, test statistic.

Answer:

- **Null hypothesis (H_0):** A default claim of no effect or no difference (e.g., $H_0: \mu = 5$). We assume it is true until evidence suggests otherwise.
- **Alternative hypothesis (H_a):** The claim we seek evidence for (e.g., $H_a: \mu > 5$). It is what we hope to support.
- **Test statistic:** A numerical value computed from sample data that measures how far the sample is from H_0 . Common examples: t-statistic, z-statistic, χ^2 -statistic, F-statistic.

The decision to reject H_0 is based on whether the test statistic falls in the **critical region**.

Q3. Define p-value and distinguish its interpretation for one-tailed vs. two-tailed tests.

Answer:

The **p-value** is the probability of obtaining a test statistic at least as extreme as the observed value, **assuming H is true**.

- **One-tailed test:** H specifies a direction (e.g., $>$). The p-value is the area in **one tail** of the distribution.
- **Two-tailed test:** H is non-directional (e.g., $\neq$). The p-value is **twice the area** in the more extreme tail.

Decision rule: If $p \leq \alpha \rightarrow$ **reject H** ; if $p > \alpha \rightarrow$ **fail to reject H** .

Q4. What is the level of significance? Describe Type I and Type II errors.

Answer:

- **Level of significance (α):** The maximum acceptable probability of making a Type I error. Commonly set at 0.05 (5%) or 0.01 (1%).

Error Type	Definition	Probability
Type I Error (False Positive)	Rejecting H when it is actually true	
Type II Error (False Negative)	Failing to reject H when it is actually false	
Power	Correctly rejecting a false H	$1 - \beta$

Trade-off: Decreasing α (stricter test) increases β (more chance of missing a real effect). Increasing sample size can reduce both.

Q5. Define t-test and F-test and explain their main purposes.

Answer:

t-test: - Tests hypotheses about **means** when the population standard deviation is unknown - Uses the **t-distribution** with $n-1$ degrees of freedom - Types: one-sample, two-sample (pooled or Welch's), paired - Example: Testing if mean weight of cats differs from 3.5 kg

F-test: - Compares **variances** — the ratio of two sample variances follows an F-distribution - Also the basis of **ANOVA** — tests whether group means differ

by comparing between-group variance to within-group variance - $F = \text{between-group variance} / \text{within-group variance}$

Q6. Explain chi-square goodness-of-fit test using `chisq.test()`, including the `p=` argument for non-uniform proportions.

Answer:

The **chi-square goodness-of-fit test** checks whether observed frequencies match an expected distribution.

- H : Data follows the specified distribution
- $\chi^2 = \sum (O - E)^2 / E$ with $df = k - 1$ (k = number of categories)

```
# Default: uniform distribution (equal expected proportions)  
chisq.test(c(30, 25, 20, 25))
```

```
# Non-uniform expected proportions using p= argument  
chisq.test(c(30, 25, 20, 25), p = c(0.4, 0.3, 0.2, 0.1))
```

The `p=` argument specifies the expected proportions for each category (must sum to 1).

Q7. Describe the step-by-step process in R for performing a chi-square test of independence.

Answer:

1. **Create contingency table:** Use `table()` or `matrix()`
2. **Check expected frequencies:** All cells should have expected frequency ≥ 5 for the test to be reliable
3. **Run the test:** `chisq.test(ct)`
4. **Interpret p-value:** If $p < \alpha \rightarrow$ reject H_0 of independence (variables are associated)
5. **Examine residuals:** Identify which cells drive the association

```
ct <- table(mtcars$cyl, mtcars$am)  
chisq.test(ct)  
chisq.test(ct)$expected      # check expected frequencies  
chisq.test(ct)$residuals     # standardized residuals (|residual| > 2 is notable)
```

Q8. Define regression analysis and its uses.

Answer:

Regression analysis models the relationship between a **response variable (Y)** and one or more **predictor variables (X)**. It is used for:

- **Prediction:** Estimating Y for new values of X
- **Quantifying effects:** Measuring how much Y changes per unit change in X
- **Understanding relationships:** Identifying which variables significantly affect Y
- **Forecasting:** Predicting future outcomes based on historical data

Output includes regression coefficients (β), R^2 , p-values, and confidence intervals.

Q9. List and explain the four key assumptions for regression model validity.

Answer:

1. **Linearity:** The relationship between X and Y is linear. Check with a residuals vs fitted plot — residuals should scatter randomly around zero.
2. **Independence:** Observations are independent of each other. Violated in time-series data with autocorrelation.
3. **Homoscedasticity:** Residuals have constant variance across all fitted values. Check with a scale-location plot — points should spread evenly.
4. **Normality of residuals:** Residuals are approximately normally distributed. Check with a Q-Q plot — points should follow a straight diagonal line.

Violations require transformations (e.g., $\log(Y)$) or alternative models (e.g., generalized linear models).

Q10. Describe simple linear regression and the role of response vs. predictor variables.

Answer:

Simple Linear Regression (SLR) models the relationship between one predictor and one response variable:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where: - **Y** = Response variable (dependent) — what we predict (e.g., mpg) -
- **X** = Predictor variable (independent) — what we use to predict (e.g., weight)
- β_0 = Intercept (value of Y when X = 0) - β_1 = Slope (change in Y per unit change in X) - ϵ = Random error term

Coefficients are estimated by **Ordinary Least Squares (OLS)**, which minimizes $\sum(Y_i - \hat{Y}_i)^2$.

Q11. Describe logistic regression with a suitable example. How is it implemented in R?

Answer:

Logistic regression models a **binary response variable** (0/1, Yes/No). Instead of predicting Y directly, it models the **log-odds**:

$$\log(p / (1-p)) = \beta_0 + \beta_1 X$$

The **sigmoid function** maps predictions to [0,1]. Output is a probability; classified as 1 if $p > 0.5$.

Example: Predicting whether a tumour is malignant (1) or benign (0) based on cell characteristics.

```
# Implementation in R
log_model <- glm(Class_bin ~ Cl.thickness + Cell.size + Cell.shape,
                  data      = bc,
                  family    = binomial(link = "logit"))
summary(log_model)

# Odds ratios
exp(coef(log_model))

# Predict probabilities
predict(log_model, type = "response")
```

Q12. What is one-way ANOVA? Differentiate between one-way and two-way ANOVA.

Answer:

One-way ANOVA tests whether the means of **3 or more groups** differ across one categorical factor.

- H_0 : All group means are equal ($\mu_1 = \mu_2 = \dots = \mu_k$)
- $F = \text{between-group variance} / \text{within-group variance}$

Comparison:

Feature	One-way ANOVA	Two-way ANOVA
Factors	1 categorical factor	2 categorical factors

Feature	One-way ANOVA	Two-way ANOVA
Tests	Main effect of factor A	Main effects of A and B + interaction A×B
R syntax	<code>aov(Y ~ A)</code>	<code>aov(Y ~ A * B)</code>
Example	Yield by fertilizer type	Yield by fertilizer AND watering schedule

Interaction in two-way ANOVA: when the effect of factor A depends on the level of factor B. If the interaction term is not significant ($p > 0.05$), interpret main effects independently.

```
# One-way
aov_one <- aov(Sepal.Width ~ Species, data = iris)
summary(aov_one)

# Two-way with interaction
aov_two <- aov(Length ~ AirBags * Man.trans.avail, data = Cars93)
summary(aov_two)
```

End of Complete Q&A — R Programming Experiments 1-4